

Zero Trust Architecture Sandbox: Implementation and Evaluation Report

Minsoo Ahn

CSE 487 T92

Student ID: 114743792

Instructor: Antonino Mione

May 11, 2026

Abstract—This report describes a hands-on implementation of a Zero Trust Architecture (ZTA) using an open-source stack on a local Kubernetes cluster. The system follows NIST SP 800-207 [1] principles with Istio as the policy enforcement point, OPA as the policy decision point, and Keycloak as the identity provider. Five security scenarios are validated: lateral movement defense, JWT forgery and tampering, context-based access control, JWT claim-based authorization, and device posture gating. All 18 test cases pass in end-to-end runs, and evidence is captured through automated test logs and OPA decision logs. Performance measurements show an average latency increase of 1.53 ms and a 22.3% throughput reduction under single-connection load tests. The results demonstrate that ZTA enforces per-request, context-aware authorization and workload identity controls that traditional firewalls and VPNs do not provide, while introducing a measurable but acceptable overhead for a security-focused environment.

I. INTRODUCTION

Modern breaches commonly follow a two-stage pattern: credential theft followed by lateral movement inside the network. Traditional perimeter controls such as firewalls and VPNs can block unauthenticated ingress but provide limited visibility and control for east-west traffic between workloads. Zero Trust Architecture changes the assumption from “trusted internal” to “verify every request” [1], [2]. This project implements a ZTA sandbox to validate that request-level identity and policy enforcement can block a set of representative attacks while remaining practical to operate on a small cluster.

II. METHODOLOGY

A. Environment and Stack

The sandbox runs on Minikube with Istio 1.28.3 for service mesh enforcement, OPA for policy evaluation, and Keycloak for JWT issuance. A simple Flask frontend and backend are deployed as separate services. Security policies are applied through Istio AuthorizationPolicy and PeerAuthentication resources, and Rego policies in OPA.

B. Policy Design

Four independent layers enforce access decisions:

- mTLS workload identity for pod-to-pod traffic.
- SPIFFE allowlist to restrict backend access to the frontend service account.
- JWT signature validation using Keycloak JWKS.

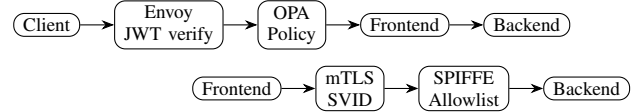


Fig. 1. Request flow and enforcement layers. Top row: north-south path (Client → Envoy JWT verify → OPA → Frontend → Backend). Bottom row: east-west path (Frontend → mTLS SVID → SPIFFE Allowlist → Backend).

TABLE I
THREAT MODEL: STRIDE MAPPING AND ATTACKER PROFILES PER SCENARIO

Sc.	STRIDE	Attacker profile (position / credentials)	Mitigating control
A	Spoofing	Internal pod / no workload identity	mTLS STRICT + SPIFFE allowlist
B	Tampering, Spoofing	External / forged or tampered JWT	JWKS signature verification
C	Elevation of Privilege	Authenticated external / unsecured role header	OPA role + method + path rule
D	Spoofing	Authenticated external / signed JWT, wrong role claim	Keycloak JWT claim + OPA
E	Elevation of Privilege	Authenticated external / valid admin JWT, risky device	OPA device posture gate

- Context-aware authorization in OPA (role, method, path, posture).

Figure 1 illustrates both the north-south and east-west enforcement flows.

C. Threat Model and Scenarios

Five scenarios represent realistic attack paths across north-south and east-west traffic. Table I maps each scenario to a STRIDE threat category, the attacker’s assumed position and credential scope, and the mitigating policy control.

D. Scenario Selection Rationale

The scenarios were chosen to reflect common attacker behavior observed in real microservice environments and to map cleanly to NIST SP 800-207 tenets. They are representative, repeatable on a local cluster, and span both north-south and east-west traffic paths without depending on rare or exotic conditions.

TABLE II
OPA REGO ALLOW RULES (SIMPLIFIED)

Scenario	Conditions (conjunction)	Decision
A (E-W)	<code>source.principal = frontend-sa ∧ method = GET ∧ path $\not\subseteq$ /api/admin</code>	allow
C	<code>role header = "user" ∧ method = GET ∧ path $\not\subseteq$ /api/admin</code>	allow
D (admin)	<code>jwt.realm_access.roles $\ni$ "admin" ∧ posture_ok</code>	allow
D (viewer)	<code>jwt.realm_access.roles $\ni$ "viewer" ∧ method = GET ∧ path $\not\subseteq$ /api/admin</code>	allow
E (gate)	<code>X-Device-Firewall $\in$ {$\emptyset$, "enabled"} $\Rightarrow$ posture_ok = true</code>	gate
Default	otherwise	deny

TABLE III
SECURITY SCENARIO OUTCOMES

Scenario	Threat and control	Result
A	Lateral movement blocked by mTLS and SPIFFE allowlist	Blocked
B	Forged or tampered JWT rejected by JWKS verification	Blocked
C	Role + method + path enforced in OPA policy	Enforced
D	JWT claim-based authorization with Keycloak + OPA	Enforced
E	Device posture gate via policy condition	Enforced

- A: Lateral movement is a dominant post-compromise behavior and validates workload identity plus service-to-service authorization.
- B: Forged or tampered JWTs are realistic API abuses and test cryptographic authentication at the infrastructure layer.
- C: Context abuse (method/path) reflects over-privileged access in practice and demonstrates fine-grained authorization beyond identity.
- D: Claim-based authorization closes header spoofing and validates signed identity as the policy source of truth.
- E: Device posture models access from risky or unmanaged devices, showing that identity alone is not sufficient.

E. Authorization Policy Logic

Table II summarizes the core allow rules in the OPA Rego policy (package `istio.authz`). All rules default to deny; a request is allowed only when every condition in one rule is satisfied. The `jwt_has_role` helper calls `io.jwt.decode` to extract `payload.realm_access.roles` without re-verifying the signature, relying on Istio’s prior JWKS verification at the mesh layer.

III. RESULTS

A. Scenario Outcomes

All 18 test cases pass in the final validation run. Table III summarizes the scenario outcomes.

B. Scenario Details

A: Lateral movement (north-south and east-west). The north-south baseline confirms unauthenticated requests are denied, while a demo header allows access to illustrate why header-only identity is weak. For east-west movement, a rogue pod without a sidecar fails mTLS, and a pod with the wrong ServiceAccount is denied by the SPIFFE allowlist. Direct pod IP access is also blocked. These tests validate workload identity and service-to-service authorization as separate gates.

B: JWT forgery and tampering. A forged JWT is rejected because no valid principal is established at the mesh layer. A tampered payload with the original signature fails cryptographic verification, returning a 401. A valid Keycloak-issued JWT is accepted and passes to OPA for authorization.

C: Context-based access control. This scenario demonstrates that identity is not enough: role, method, and path must be consistent. A user role can read `/api/data` but is denied on `/api/admin` and all write operations, while an admin role can write. This uses header-based roles as a deliberate baseline for contrast with Scenario D. In the final configuration, JWT authentication is still enforced; the header-based rules illustrate why signed claims are needed even when a request is authenticated.

D: JWT claim authorization. Authorization is driven by signed claims in the JWT. An admin token can access admin endpoints, while a viewer token is limited to read-only paths. This closes header spoofing and demonstrates a cryptographically bound identity source of truth.

E: Device posture gating. Even with a valid admin JWT, access is denied when the device posture signal indicates risk. The demo uses the `X-Device-Firewall` header to model a health signal and illustrates how posture can be integrated into policy decisions.

Full test case results for all scenarios are shown in Table IV.

C. Operational Evidence

OPA decision logs confirm allow and deny outcomes for each scenario, and the automated `make test-all` run provides a consolidated PASS summary. The evidence directory includes detailed logs for JWT forgery, lateral movement attempts, and device posture simulation. Figure 2 shows the Kiali workload graph confirming mTLS enforcement across active service paths.

D. Performance

Performance testing uses Fortio with 200 requests on a single persistent connection. The load parameters were chosen to isolate the pure policy evaluation cost—specifically the OPA gRPC round-trip latency and JWT payload parsing—without variance from connection-establishment overhead or scheduler contention under concurrent load. Minikube runs inside Docker Desktop on WSL2, where CPU resources are shared with the host OS; a single-connection sequential load avoids throttling artifacts that would otherwise conflate authorization latency with resource contention. The throughput figures therefore represent sequential-request capacity rather than peak

TABLE IV
COMBINED TEST MATRIX: ALL SCENARIOS A–E (18 CASES)

Scenario	Case	Request signals	Expect	Result
A	A-NS-1	GET /api/admin, no identity	403	403
	A-NS-2	GET /api/admin, header role:admin	200	200
	A-EW-1	Rogue pod (no sidecar) to backend service	403/000/503	403
	A-EW-2	Wrong ServiceAccount (backend-sa) to backend	403	403
	A-EW-3	Rogue pod to backend pod IP:8080	403/000/503	503
B	B-1	Forged JWT (invalid signature) to /api/admin	403	403
	B-2	Tampered real JWT payload to /api/admin	401	401
	B-3	Valid Keycloak JWT to /api/admin	200	200
C	C-1	role:user, GET /api/data	200	200
	C-2	role:user, GET /api/admin	403	403
	C-3	role:user, POST /api/write	403	403
	C-4	role:admin, POST /api/write	200	200
D	D-1	Admin JWT, GET /api/admin	200	200
	D-2	Viewer JWT, GET /api/data	200	200
	D-3	Viewer JWT, GET /api/admin	403	403
	D-4	Viewer JWT, POST /api/write	403	403
E	E-1	Admin JWT + X-Device-Firewall: enabled	200	200
	E-2	Admin JWT + X-Device-Firewall: disabled	403	403

TABLE V
PERFORMANCE IMPACT (200 REQUESTS, SINGLE CONNECTION)

Metric	Baseline	ZTA	Delta
Avg latency	5.33 ms	6.86 ms	+1.53 ms (+28.6%)
QPS	187.5	145.7	−41.8 (−22.3%)
p99 latency	8.5 ms	10.5 ms	+2.0 ms (+23.5%)

concurrency. Table V summarizes the aggregate overhead relative to a baseline with no Istio policies applied.

Figure 3 visualizes the latency difference between baseline and ZTA.

Table VI breaks down per-scenario timing. Blocked cases measure policy-gate cost only (the request is denied before reaching the application), while allowed cases include full end-to-end handling; the two categories should be compared separately.

IV. DISCUSSION

A. Security Value vs Traditional Controls

The implementation demonstrates that per-request authorization at the infrastructure layer can block attack paths that

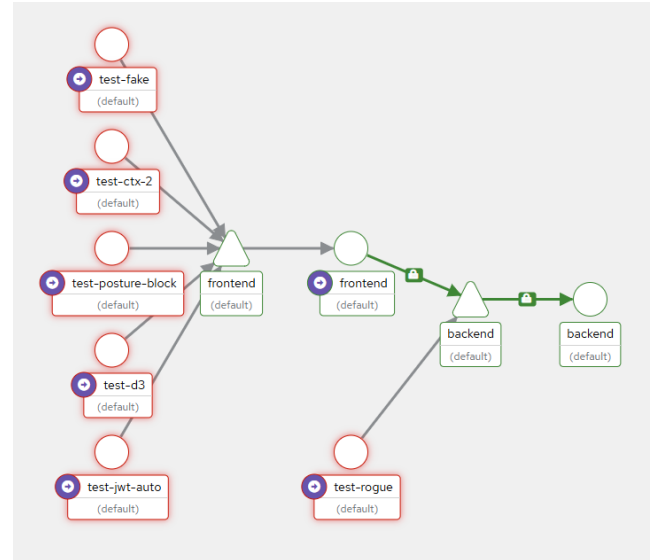


Fig. 2. Kiali Workload Graph captured during the make for_pdf test run, showing one representative request per scenario (A–E). The mTLS padlock icon on the frontend→backend edge confirms STRICT mutual authentication is active. Blocked paths (rogue pod, forged JWT) terminate at the ingress and do not appear as edges to the backend, illustrating that the enforcement layers are working as intended.

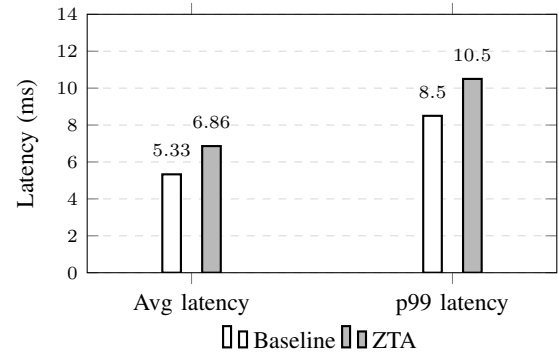


Fig. 3. Latency overhead introduced by ZTA policies (Fortio, 200 requests, single connection). QPS: baseline 187.5, ZTA 145.7 (−22.3%).

traditional perimeter controls miss. Even with valid credentials, requests are denied when policy context is violated, and internal lateral movement is blocked by workload identity checks.

B. VPN vs ZTA

WireGuard [3] provides low-overhead transport security but does not enforce request-level authorization or workload identity. ZTA adds these controls with a measurable overhead, so the most practical deployment is layered: VPN for secure entry and ZTA for service policy enforcement.

V. LIMITATIONS

Device posture is simulated through a client-provided header, and the OPA ext-authz channel operates over plain-text gRPC on port 9191 rather than mTLS. This is a de-

TABLE VI
PER-SCENARIO LATENCY (FORTIO, 200 REQUESTS): ALLOW CASES ARE
END-TO-END; BLOCK CASES MEASURE POLICY GATE ONLY.

Case	Type	Request signal	Avg (ms)	p99 (ms)	Code
A1	Allow	Admin JWT, GET /api/admin	5.94	7.89	200
A2	Allow	Viewer JWT, GET /api/data	6.63	8.80	200
A3	Allow	role:user, GET /api/data	6.58	9.00	200
A4	Allow	Admin JWT + device firewall enabled	6.22	7.94	200
B1	Block	No identity, GET /api/admin	2.02	4.00	403
B2	Block	Forged JWT, GET /api/admin	2.36	5.00	403
B3	Block	Tampered JWT, GET /api/admin	2.00	4.00	401
B4	Block	Admin JWT + device firewall disabled	2.91	5.50	403

liberate architectural constraint: injecting an Envoy sidecar into the OPA pod would create a circular authorization dependency. Every Envoy sidecar in the mesh dispatches a `CheckRequest` to OPA before forwarding an application request. If OPA’s pod itself carried an injected sidecar, that sidecar’s inbound filter chain would intercept the incoming `CheckRequest` and—following the same ext-authz filter configuration— would issue a new `CheckRequest` to OPA to authorize the original authorization request. This forms an infinite loop: Envoy → OPA → (OPA’s own Envoy intercepts) → OPA → ... OPA must therefore reside outside the sidecar mesh. The plaintext channel is bounded to intra-cluster traffic by a Kubernetes NetworkPolicy (`opa-network-policy.yaml`) that restricts port 9191 to mesh-internal peers only. Header-based role rules remain in place to demonstrate weaker access controls (Scenario C) alongside the stronger JWT claim path (Scenario D). The scenario set is representative but not exhaustive, and ZTA does not eliminate risks from human error, misconfiguration, or novel low-level exploits. Sequential-connection load tests do not capture behavior under concurrent request pressure. The environment is local and does not represent production-scale latency or throughput.

VI. EXPERIENCE AND REFLECTION

A. Unexpected Outcomes

I expected the implementation to be harder, but the core ZTA flow was manageable once the stack was aligned. The most surprising result was performance overhead: the latency increase and QPS drop were higher than I anticipated, even in a small test.

B. Implementation Challenges

The initial setup and integration of Istio, OPA, and Keycloak was the most time-consuming part. JWT verification flow

and automated end-to-end tests required careful ordering and debugging. I also spent time looking for missing scenarios and thinking about bypasses not captured by the default test set.

C. Limits of ZTA and Current Practice

This project reinforced that ZTA is a strong practical approach but not perfect. It does not eliminate risks from human error, system misconfiguration, or novel low-level exploits, and it only blocks the scenarios it explicitly models. AI-assisted work helped accelerate policy iteration, but it did not remove the need for manual validation, and it reminded me that attackers can also use AI to discover new paths. ZTA feels like the best available option today, not a final solution.

D. Lessons Learned and Future Work

I learned that layered controls matter more than any single tool, and that the security-performance trade-off is real. As hardware improves, some overhead will be easier to absorb, but the dilemma remains. If I extend this work, I want to design a more efficient policy path and push enforcement closer to lower-level boundaries so that isolation holds even against unexpected paths.

VII. CONCLUSION

The sandbox validates that open-source tools can implement a realistic Zero Trust Architecture aligned with NIST SP 800-207 [1]. The system blocks five representative attack scenarios and provides clear audit evidence for each decision. The measured performance cost is acceptable for a security-focused environment, and the design illustrates how identity, context, and workload trust can replace implicit network trust.

REFERENCES

- [1] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, “Zero Trust Architecture,” National Institute of Standards and Technology, Gaithersburg, MD, NIST SP 800-207, Aug. 2020. [Online]. Available: <https://doi.org/10.6028/NIST.SP.800-207>
- [2] R. Ward and B. Beyer, “BeyondCorp: A New Approach to Enterprise Security,” *login:*, vol. 39, no. 6, pp. 6–11, Dec. 2014. [Online]. Available: <https://research.google/pubs/pub43231/>
- [3] J. A. Donenfeld, “WireGuard: Next Generation Kernel Network Tunnel,” in *Proc. Network and Distributed System Security Symp. (NDSS)*, San Diego, CA, USA, Feb. 2017. [Online]. Available: <https://www.wireguard.com/papers/wireguard.pdf>